# *Analysis and Design of Algorithms*

NUS National University of Singapore

CS3230

## *Recitation*

Week 5

**Prof Lee Wee Sun**

# Randomized Algorithms

Program behaviour determined not only by input but also by values produced by random-number generator.

- Randomization can sometimes prevent deterministic worst-case performance, e.g. randomized quicksort
- Often simpler and fast

# Probabilistic Analysis

Probabilistic analysis: analyze properties of algorithms, usually running time

- Average-case runtime for deterministic algorithm – need to know (or assume) input distribution

# Tools for Probabilistic Analysis

- Analysis means taking things apart into smaller parts, understanding the parts and putting them back together
- Main tools used for probabilistic analysis:
  - Indicator random variable
  - Linearity of expectation

# Admin

- Reminder to submit Homework 1 by Friday midnight. Submit file under Missions then Homework 1.

# MCQ

# Question 1

Randomized algorthms can be classified into:

**Las Vegas algorithm:** Always returns the correct answer but the runtime is random.

**Monte Carlo algorithm:** Constant runtime but the answer returned is not guaranteed to be correct.

Consider the following two algorithms.

**Randomized quicksort:** As described in lecture.

**Finding $\pi$:** Randomly sample $(x,y)$ $n$ times with $x$ and $y$ uniformly distributed in [-1,1] and count the fraction satisfying $x^2 + y^2 \leq 1$. Multiply the fraction by 4 and use it as the estimate for $\pi$.

Classify the two algorithms into Las Vegas and Monte Carlo algorithms.

- Randomized quicksort: Las Vegas

  Finding $\pi$: Las Vegas

- Randomized quicksort: Las Vegas

  Finding $\pi$: Monte Carlo

- Randomized quicksort: Monte Carlo

  Finding $\pi$: Las Vegas

- Randomized quicksort: Monte Carlo

  Finding $\pi$: Monte Carlo

# Solution

Quicksort always returns the correct solution, so is a Las Vegas algorithm.

Finding $\pi$ only returns an approximate solution, so is a Monte Carlo algorithm.

# Questions 2 to 5

You are one of Professor X's student and have just saved the world from Magneto. Professor X will reward you with one of $n$ new abilities.

- You are given a list of $n$ abilities. Unfortunately you decide how good an ability is without seeing it.
- You may sequentially request to see an ability.
- After seeing an ability, you have to decide whether to accept it.
- If you do not accept the ability, you may no longer select the ability in future.
- On seeing a new ability, you can choose to switch from the ability that you have previously accepted, but you will have to perform a mission for Professor X in order to change the ability.
- You want the best ability, and will always decide to switch if you see a better ability.

# Question 2

What is the worst case number of missions that you have to perform for Professor X?

- $\Theta(1)$
- $\Theta(\lg n)$
- $\Theta(n)$

# Solution

In the worst case, the abilities are ordered in increasing order, so you have to switch every time, giving $n$-1 switches after accepting the first one.

# Question 3

You decide to randomly permute the list and then go through it in sequence. What is the probability that you will switch on the $i$th item of the permuted list?

- 1
- 0.5
- $1/i$
- $1/n$

# Solution

Given the first $i$ items, we want to compute the probability that the largest item is in position $i$. As the permutation is random, the probability for the largest item being at each position is equally likely, $1/i$.

# Question 4

You decide to randomly permute the list and then go through it in sequence. What is the expected number of missions that you have to do for Professor X?

- $\Theta(1)$
- $\Theta(\lg n)$
- $\Theta(n)$

# Solution

Let $X_i$ be the indicator random variable for accepting ability $i$. Then $X = \sum_{i=1}^{n} X_i$ is the number of accepted abilities

The expected number of accepted abilities is

$$
\begin{aligned}
E[X] &= \sum_{i=1}^{n} E[X_i] \\
&= \sum_{i=1}^{n} \frac{1}{i} \\
&= H_n \\
&= \Theta(\lg n)
\end{aligned}
$$

Expectation of indicator R.V.

where $H_n$ is the n-th harmonic number

# Question 5

You decide to randomly permute the list and then go through it in sequence. Professor X can read minds. Should you be worried that he can make you do the worst case number of missions?

- Yes
- No

# Solution

As long as Professor X does not have control over the source of randomness that you use to generate the random permutation, he is not able to affect the expected number of missions that you have to take.

# Question 6

Who is the Master of Algorithms pictured below?

- Ronald Rivest
- Alan Turing
- Donald Knuth
- Tony Hoare

## Sir Anthony Hoare

Turing Award winner

- Invented quicksort
- Also invented quickselect (later lecture).
- Known for Hoare logic (for correctness), communicating sequential processes, monitor, axiomatic specification of computer language, ...

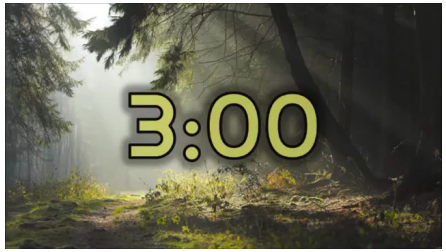
# Structured Questions

# Question 1(a)

Bucket sort has *average-case* runtime of $O(n)$ when the inputs are drawn independently from the **uniform distribution** from the interval $[0,1)$

- Bucket sort partitions [0,1) into *n* equal sized intervals (buckets).
- The inputs are then distributed into the corresponding buckets, with a linked list used to store the elements in each bucket.
- Each bucket is then sorted using *insertion sort*.
- Finally, we go through the buckets in order, listing out the elements..

What is the worst case running time of bucket sort and when does it happen?



# Solution

The worst-case running time is $\Theta(n^2)$.

The worst case happens when all the elements fall into the same bucket.

Insertion sort on a list of length $n$ takes $\Theta(n^2)$ time.

# Question 1(b)

Let $n_i$ be the number of items in bucket $i$. Argue that the average-case running time of bucket sort is

$$E[T(n)] = \Theta(n) + \sum_{i=0}^{n-1} O(E[n_i^2])$$

# Solution

The runtime consist of distributing the elements into the buckets and listing out the elements at the end ($\Theta(n)$) and the time to sort each bucket ($n_i^2$ for bucket $i$), giving

$$T(n) = \Theta(n) + \sum_{i=0}^{n-1} O(n_i^2)$$

Taking expectation

$$E[T(n)] = E[\Theta(n) + \sum_{i=0}^{n-1} O(n_i^2)]$$

$$= \Theta(n) + \sum_{i=0}^{n-1} O(E[n_i^2])$$

where we used linearity of expectation in the 2nd line

# Question 1(c)

Let

$$X_{ij} = \begin{cases} 1 & \text{if } A[j] \text{ falls in bucket } i \\ 0 & \text{otherwise} \end{cases}$$

Argue that

$$E[n_i^2] = \sum_{j=1}^{n} E[X_{ij}^2] + \sum_{1 \le j \le n} \sum_{1 \le k \le n, k \ne j} E[X_{ij} X_{ik}].$$

# Solution

We can write $n_i = \sum_{j=1}^{n} X_{ij}$

Taking expectation:

$$
\begin{aligned}
E[n_i^2] &= E\left[\left(\sum_{j=1}^{n} X_{ij}\right)^2\right] \\
&= E\left[\sum_{j=1}^{n}\sum_{k=1}^{n} X_{ij}X_{ik}\right] \\
&= E\left[\sum_{j=1}^{n} X_{ij}^2 + \sum_{1\le j\le n}\ \sum_{1\le k\le n, k\ne j} X_{ij}X_{ik}\right] \\
&= \sum_{j=1}^{n} E[X_{ij}^2] + \sum_{1\le j\le n}\ \sum_{1\le k\le n, k\ne j} E\left[X_{ij}X_{ik}\right].
\end{aligned}
$$

# Question 1(d)

Using $E[X_{ij}^2] = 1/n$ and $E[X_{ij}X_{ik}] = E[X_{ij}]E[X_{ik}] = 1/n^2$ when $k \neq j$, argue that

$$E[n_i^2] = 2 - 1/n.$$

Let

$$X_{ij} = \begin{cases} 1 & \text{if } A[j] \text{ falls in bucket } i \\ 0 & \text{otherwise} \end{cases}$$

Argue that

$$E[n_i^2] = \sum_{j=1}^{n} E[X_{ij}^2] + \sum_{1 \le j \le n} \sum_{1 \le k \le n, k \neq j} E[X_{ij}X_{ik}].$$

From prev part

Combined with part (b), this completes the proof that the average-case runtime is linear in the size of the input.

# Solution

$$
\begin{aligned}
E[n_i^2] &= \sum_{j=1}^{n} \frac{1}{n} + \sum_{1 \le j \le n} \sum_{1 \le k \le n, k \ne j} \frac{1}{n^2} \\
&= n\frac{1}{n} + n(n-1)\frac{1}{n^2} \\
&= 1 + \frac{n-1}{n} \\
&= 2 - \frac{1}{n}.
\end{aligned}
$$